

Conjugate operators for transparent, explorable research outputs

ANONYMOUS AUTHOR(S)

Charts, figures, and text derived from data play an important role in decision making, from data-driven policy development to day-to-day choices informed by online articles. Making sense of, or fact-checking, outputs means understanding how they relate to the underlying data. Even for domain experts with access to the source code and data sets, this poses a significant challenge. In this paper we introduce a new program analysis framework which supports interactive exploration of fine-grained I/O relationships directly through computed outputs, making use of dynamic dependence graphs. Our main contribution is a novel notion in data provenance which we call *related inputs*, a relation of mutual relevance or “cognacy” which arises between inputs when they contribute to common features of the output. Queries of this form allow readers to ask questions like “What outputs use this data element, and what other data elements are used along with it?”. We show how [Jonsson and Tarski’s](#) concept of conjugate operators on Boolean algebras appropriately characterises the notion of cognacy in a dependence graph, and give a procedure for computing related inputs over such a graph.

To demonstrate the approach in practice, we present a functional programming language called [OurLanguage] which automatically enriches visual outputs with interactions supporting related inputs and similar fine-grained queries, and show how these can aid comprehension using realistic examples. We show how to obtain more informative, contextualised queries via projection operators which factor out data contributed by specific inputs or demand associated with specific outputs. We also show how to derive a *related outputs* operator, capturing a dual cognacy relation between output features whenever they compete for common input data. Composed with suitable projections, this recovers a prior approach to the problem of *brushing and linking* in data visualisation based on Galois connections and execution traces. As well as generalising and extending this prior work, the graphical approach presented in this paper performs better (computationally) on most examples and is significantly simpler to implement, and thus easier to deploy to other languages.

1 INTRODUCTION: TOWARDS TRANSPARENT RESEARCH OUTPUTS

Whether formulating a national policy or simply deciding what groceries to buy, we increasingly rely on the charts, figures and text created by scientists and journalists. Interpreting these visual and textual summaries is essential to making informed decisions. However, most of the artefacts we encounter are *opaque*: they are unable to reveal anything about how they relate to the data they were derived from. Whilst one could in principle try to use the source code and data sources to reverse engineer some of these relationships, this requires substantial expertise, as well as valuable time spent away from the “comprehension context” in which we encountered the output in question. These difficulties are only compounded when the information presented draws on multiple data sources, such as medical meta-analyses [[DerSimonian and Laird 1986](#)], ensemble models in climate science [[Murphy et al. 2004](#)], or queries that span multiple database tables [[Selinger et al. 1979](#)]. Even professional reviewers may lack the resources or inclination to embark on such an activity. Perhaps more often than we would like, we end up taking things on trust.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Association for Computing Machinery.

2475-1421/2024/2-ART1 \$15.00

<https://doi.org/>

With traditional print media, a “disconnect” between outputs and underlying data is unavoidable. For digital media, other options are open to us. One way to improve things is to engineer visual artefacts to be more “self-explanatory” so they can reveal the relationship to the underlying data to an interested user. Consider the histogram in Figure 1 showing urban population growth in Asia over a 10-year period [Bremer and Ranzijn 2015]. There are many questions a reader might have about what this chart represents — in other words, how visual elements map to underlying data. Do the points represent individual cities? What does the colour scheme indicate? Which of the points represent large cities or small cities? Legends and other accompanying text can help, but some ambiguities inevitably remain. The entire value proposition of a summary after all is that it presents the big picture at the expense of certain detail.

Bremer and Ranzijn implemented an interactive feature that allow a user to explore some of these questions themselves *in situ*, i.e., directly from the chart. By selecting an individual point, they are able to bring up a view showing the data that point was calculated from. In this case the highlighted dot represents data for Chiang Mai and the plotted value of 10.9% was derived from an increase in population density from 4,416 to 5,010 people per sq. km.

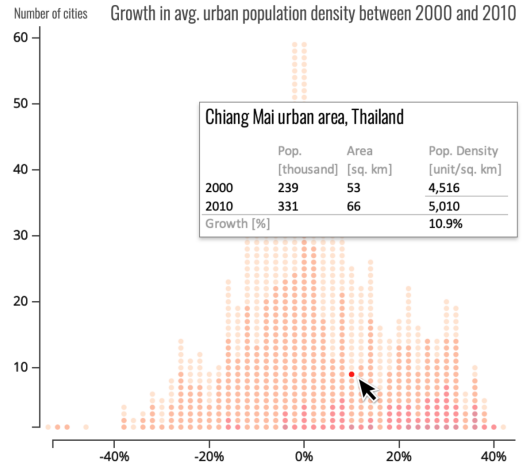


Fig. 1. Hand-crafted data-transparent visualisation [Bremer and Ranzijn 2015]

1.1 Data transparency as PL infrastructure

Features like these are valuable comprehension aids but are laborious to implement by hand. They also require the author to anticipate the queries a user might have and do not readily generalise: for example Bremer and Ranzijn’s visualisation only allows the user to select one point at a time.

For these reasons, there has been recent interest in treating data transparency and exploration as a programming language infrastructure problem, baking dependency metadata directly into outputs so that interactive queries of this sort can be supported automatically. The author of the content no longer has to concern themselves with these tricky features, and the consumer of the content now has the best of both worlds: the ability to explore detail on an as-needed basis in the context of the high-level perspective offered by the visual summary. The relationship to data can be incrementally discoverable, revealed as the user interacts with the parts of the output they care about, ideally with some kind of formal guarantee that the revealed data is in some sense “minimal and sufficient”.

In data visualisation it has long been recognised that when users interact with visual elements by clicking or lassoing, the intention is typically to manipulate the underlying data; this has led to various techniques for inverting selections in visual space to obtain selections in the input data space [North and Shneiderman 2000; Heer et al. 2008]. Psallidas and Wu [2018] were the first to propose techniques based on fine-grained data provenance; for a relational visualisation language, they describe how to “backward trace” from output selections to input selections, and then “forward trace” to find related output selections in other views, to support a popular feature called *linked brushing*. Perera et al. [2022] implemented a similar approach for a general-purpose functional language using a pair of dependency-tracking operators, written here as ∇ (demands) and $\blacktriangle$ (suffices).

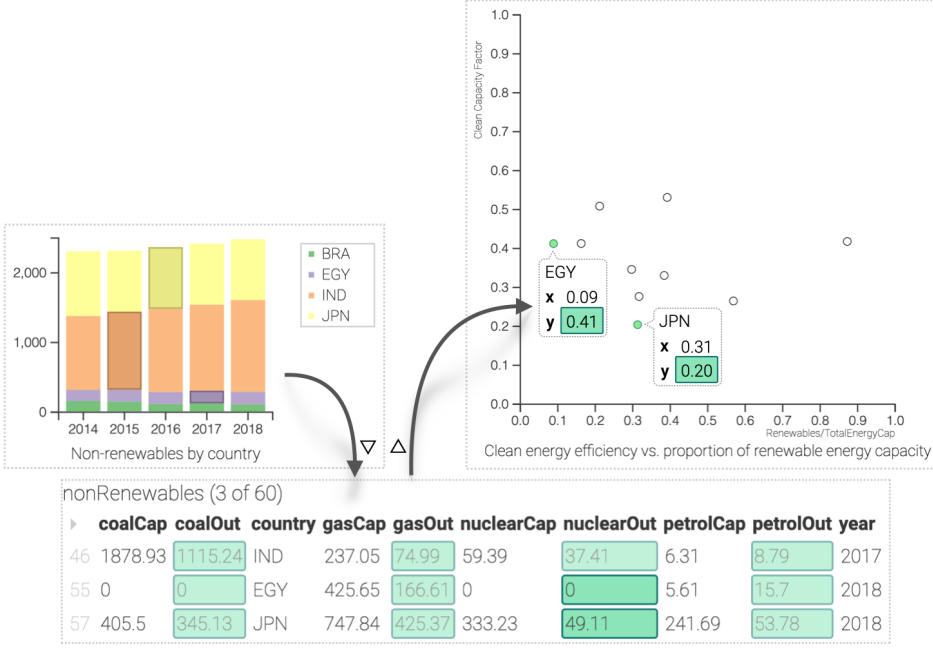


Fig. 2. Data-transparent visualisations associate visual selections with demanded data

for), relating specific output and input selections. The formal guarantee that input selections are “minimal and sufficient” was captured by ∇ and $\blacktriangle$ forming a *Galois connection*, meaning they enjoy the following relationship of (near) mutual reciprocity:

$$\nabla(Y) \leq X \iff Y \leq \blacktriangle(X)$$

(Here $\leq$ is the inclusion order over sets X, Y of data elements.) This says that if outputs Y demand inputs X' , then X' is the smallest set of inputs which suffices for outputs which include Y ; and conversely that if inputs X suffice for Y' , then Y' is the greatest set of outputs whose demand is included by X . (A Galois connection in this sense simply relaxes the usual isomorphism laws to inequalities.) Galois connections lend themselves well to bidirectional reasoning about resource consumption because the $\blacktriangle$ direction (sufficiency) can be interpreted in terms of (fine-grained) reproducibility: a way of verifying that a supplied data set (perhaps a subset of a larger one) is in fact sufficient to reproduce a particular feature of a complex output.

We call this broad idea of fine-grained *in situ* provenance queries *data transparency*, and illustrate this in Figure 2 using our implementation, which builds on these provenance-inspired approaches. Consider the two charts at the top of the figure. As with the histogram, there are many questions even a domain expert might have about these charts. For example, in renewable energy one must be careful not to confuse *capacity* with *output* – which of these is being shown in the bar chart? What kinds of energy count as “non-renewable”? In the scatter plot, the label on the x-axis suggests that it represents some kind of part-to-whole ratio (renewables to total energy capacity, perhaps), whereas the y-axis purports to represent the “capacity factor” of clean energy sources. But these labels are little more than informal commentary. Moreover there are *two* outputs here and so the question of if and how they might be related also arises.

A system with automated data transparency allows the user to investigate questions like these by interacting directly with the artefacts. By selecting an interesting part of the output, they can obtain a view of the relevant data. On the left, the three bar segments highlighted with a dark border indicate such a selection; the table underneath shows the demanded data (computed using our version of the ∇ operator) and we are also able to automatically select any outputs in the other chart which demand any of the same data using an other operator Δ (“demanded by”). The overlapping demand, which we call the *mediating input*, is highlighted using a darker shade of green in Figure 2. Perera et al defined such a Δ operator as the De Morgan dual of the “suffices for” operator $\blacktriangle$; one of our contributions here is to show that Δ can be computed in a more direct fashion using dependence graphs.

1.2 Contributions

In this paper, we develop the idea of data transparency in two new directions. First we approach the problem in a language-independent way, using *dynamic dependence graphs*. This separates the definition of queries over the I/O dependencies for a particular program from the problem of generating a dependence graph for that program. Dependence graphs are a proven technique which have been used extensively for program slicing (mainly of imperative programs [Ferrante et al. 1987]), but have not yet been applied to data analyses based on adjoint pairs of operators (such as ∇ and $\blacktriangle$). These analyses (e.g. Ricciotti et al. [2017]) have instead utilised a bidirectional interpreter that builds an execution record in the form of a *trace* during a forwards execution, folding that trace back into a program slice during a backwards execution. This method is quite inefficient and non-trivial to prove correct; instead, we show that a graph-based approach factors the problem into a (language-specific) graph-building interpreter plus (language-agnostic) adjoint analyses over the graph. This partitioning reduces the implementation burden and improves performance compared to approaches based on bidirectional interpretation.

Second, we introduce a new kind of provenance query called *related inputs*, a relation of mutual relevance which arises between inputs when they contribute to common features of the output. Users are able to formulate queries like “What outputs use this data element, and what other data elements are used along with it?”. Such questions explore *cognacy* (common ancestry) in a dependence graph G , and are formally dual to the “related outputs” feature shown in Figure 2, which can be understood as cognacy in G^{-1} . In the graphical setting, these are both easy to compute, since they amount to the two ways of composing reachability in G with its relational converse.

Roadmap. § 2 presents an overview of our approach and language, and motivates the idea of “related inputs” queries from an end-user perspective. The rest of the paper is organised as follows:

- § 3 defines a new program analysis framework over dynamic dependence graphs, introducing the cognacy operator $\nabla\Delta$ (“related inputs”) and unpacking the intuitive relationship between its two components ∇ (“demands”) and Δ (“demanded by”) in terms of Jonsson and Tarski’s notion of conjugate operators over Boolean algebras. We also introduce the dual cognacy operator $\Delta\nabla$ (“related outputs”) and explain the relationship to Galois connections: f and g being conjugate is equivalent to f and the De Morgan dual of g forming a Galois connection. We give procedures for computing ∇_R and $\blacktriangle_R$ over a directed graph with reachability relation R .
- § 4 defines a core functional language with an operational semantics that pairs every result with a dynamic dependence graph suitable for computing ∇ and Δ . We show how to represent (parts of) values as vertices in the graph.
- § 5 compares the performance of our implementation based on graphs and conjugates with an implementation due to Perera et al. [2022] based on traces and Galois connections, contrasting

the overhead of building trace vs. building dependence graphs, computing ∇ and Δ over each, and the relative implementation burden of the two approaches.

- § 6 reviews related work in more detail, including program slicing and DDGs.
- § 7 wraps up with a discussion of some limitations. In particular, the sort of transparency infrastructure proposed in this paper is purely extensional and falls short of providing full explanations of how output parts are related to input parts. We discuss extending this to more “intensional” forms transparency and propose some other ways in which the present system could be improved.

2 OVERVIEW OF APPROACH AND SOLUTION

As outlined in § 1, the idea of data-transparent outputs is to enrich computed content with interactions that allow the user to query relationships between data sources and visualisations and other outputs *in situ*, i.e. within the “comprehension context” in which these sorts of questions naturally arise. Crucially, we want to make this feature automatic. Hand-crafted efforts like [Bremer and Ranzijn \[2015\]](#)’s are not only labour-intensive, but involve manually embedding metadata about the relationship between visual outputs and inputs into the same program. The validity of this metadata is fragile: whenever the visualisation or data analysis logic changes, the relationships between inputs and outputs also change. By shifting the responsibility for gathering this information into the programming language, the author or data scientist can concern themselves purely with analysis and visualisation, and defer the provision of these transparency features to the infrastructure used to implement and host the visualisation.

However, computing the bidirectional dependency information needed to realise this idea is costly both in terms of performance and implementation burden. For example, the approach taken by [Perera et al. \[2022\]](#) to implement a “related outputs” analysis like the one shown in Figure 2 requires two separate analyses: a backwards “demands” analysis to determine data needed by an output selection, and a forwards “suffices for” analysis whose De Morgan dual determines any parts of the other chart that also need any of that data. These analyses need to be defined separately and each depends on details of the language, imposing a significant burden on the implementor. Moreover each runs over an execution record or trace of the entire computation regardless of whether all of the trace is relevant.

2.1 Conjugate Operators over Dependence Graphs

Our insight in this paper is that dependence graphs provide an alternative, more language-agnostic foundation for data transparency that both improves performance and reduces the implementation burden. Moreover, certain problems are much easier to formulate in the graphical setting. For example “related outputs” simply amounts to computing common ancestry, or *cognacy*, in G , which turns out to be nicely captured by the concept of “conjugate” operators [[Jonsson and Tarski 1951](#)] between Boolean algebras. Switching to the concrete setting of powersets, $f : \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$ and $g : \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$ are *conjugate* iff:

$$f(X) \cap Y = \emptyset \iff X \cap g(Y) = \emptyset$$

The canonical example of conjugate operations are the image and preimage functions $R[\cdot]$ and $R^{-1}[\cdot]$ for a relation $R \subseteq X \times Y$. It is not hard to see that if $R^{-1}[Y']$ overlaps with X' where $X' \subseteq X$ then necessarily $R[X']$ overlaps with Y' where $Y' \subseteq Y$, and vice versa. Conjugate pairs are also closely related to Galois connections; if f and g are conjugate, then f and the De Morgan dual of g form a Galois connection (which will be explained in more detail in § 3).

If we now take R to be reachability in G , then our setting technically only requires a single operator ∇_R (a “demands” relation) to support all the required queries. Its adjoint $\blacktriangle_R$ is readily

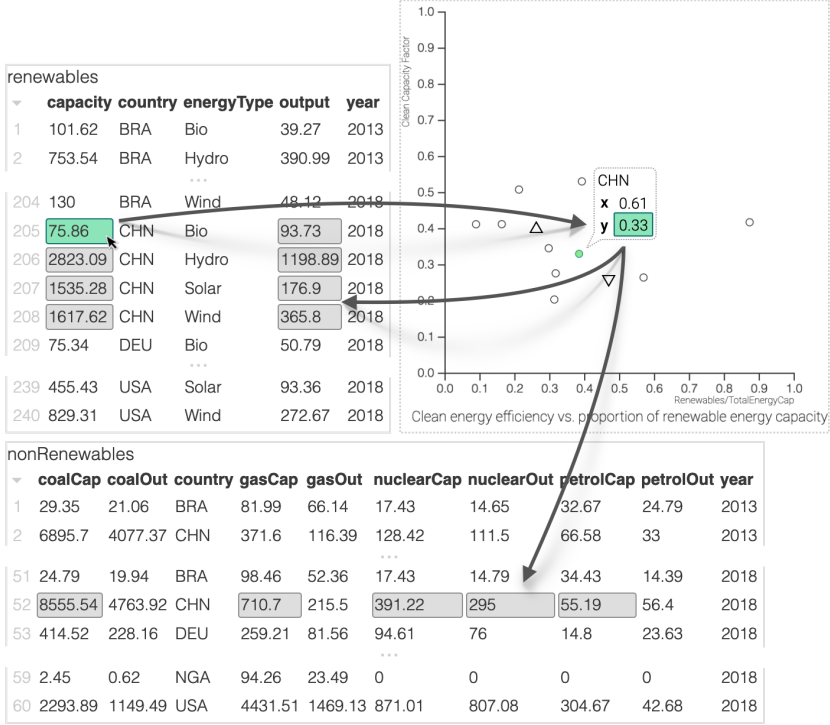


Fig. 3. Related inputs are cognate (have a common output dependency)

computed as $\nabla_{R^{-1}}$ and its conjugate Δ_R as the De Morgan dual of $\nabla_{R^{-1}}$. It is well-known that extensionally any one of these is enough to determine each other, but in the graph setting we can also get the benefit of efficient procedures for all of them in terms of reachability. In particular the “related outputs” relation is simply given by $\nabla_{R_3R^{-1}}$. Since going via the De Morgan dual can be inefficient, we also give a direct implementation of Δ_R which improves on the derived version in those cases.

2.2 Related Inputs

Our second insight in this paper is that the graphical setting makes it straightforward to compose reachability in G and with its converse the other way around, producing another provenance query called *related inputs* ($\nabla\Delta$). Figure 3 illustrates related inputs, again using our implementation. Here the user starts from one of the input data sets, expressing interest in the bioenergy capacity of China for 2018 by moving their mouse over the appropriate cell in the table (Figure 3 step 1, green selection). The system generates two further selections in response. First, any scatter plot elements that demand the selected input are given a similar green highlight; here just one point is highlighted, and in fact only the y coordinate of that point (step 2, tooltip). We call this output selection the *mediating output* because of its role in establishing a link between otherwise unrelated inputs.

Finally, any other inputs demanded by the mediating output (i.e. that were needed to compute the y coordinate of that point) are highlighted in grey (steps 3a and 3b). (The grey is used to visually distinguish the *answer* to the query from the selection that initiated the query in the first place.) The result of a related inputs query can be thought of as a unit of comprehension or unit of reuse.

```

295 1  let isCountry name x = name == x.country;
296 2      isYear year x = year == x.year;
297 3
298 4  let plot year countries =
299 5      let rens = filter (isYear year) renewables;
300 6          nonRens = filter (isYear year) nonRenewables;
301 7      let plotCountry country =
302 8          let rens' = filter (isCountry country) rens;
303 9              rensOut = sum (map (fun x → x.output) rens');
304 10              rensCap = sum (map (fun x → x.capacity) rens');
305 11              x = head (filter (isCountry country) nonRens);
306 12              nonRensCap = x.nuclearCap + x.petrolCap + x.gasCap + x.coalCap
307 13          in {
308 14              x: rensCap / (rensCap + nonRensCap),
309 15              y: (rensOut + x.nuclearOut) / (rensCap + x.nuclearCap)
310 16          }
311 17      in map plotCountry countries
312 18
313 19  in ScatterPlot {
314 20      caption: "Clean energy efficiency vs. proportion of renewable energy capacity",
315 21      data: plot 2018 [ "BRA", "CHN", "DEU", "FRA", "EGY", "IND", "JPN", "MEX", "NGA", "USA" ],
316 22      xlabel: "Renewables/TotalEnergyCap",
317 23      ylabel: "Clean Capacity Factor"
318 24  }

```

Fig. 4. Source code for scatter plot in Figure 2

It picks out all the relevant data needed to understand or make use of the original selected input, and does so in a way that identifies the common output elements that explain why the inputs are related. Sometimes a single input is used in many different outputs, or different features of a single output, in which case the query result can be quite noisy (because lots of inputs turn out to be related); in § 3 we also show how selectively projecting away irrelevant parts of the mediating output can refine the query and produce a more precise answer. The same approach can be used to discard irrelevant mediating inputs to obtain more precise results in a related outputs query.

2.3 [OurLanguage]: A Data-Transparent Programming Language

We implement the abstract framework outlined above in a programming language called [OurLanguage], a pure functional language with a runtime implementing conjugate dependency operators. From the programmer's perspective the language is unremarkable; Figure 4 shows the source code for the scatter plot example from Figure 2. The distinguishing feature of the language is a special semantics that interprets a program not as a plain output but as an output with a built-in query engine supporting the transparency features shown above. The author of a visualisation just expresses their chart as a pure function of the inputs, via a set of data types for common visualisations; a d3.js front end automatically enriches the rendered outputs with support for visual selection and various selection-based queries.

In § 3 we set out the formal graph framework and in § 4 show how the [OurLanguage] interpreter implements that abstract framework. Our implementation is open source and available at [anonymised], which also has several interactive demos.

3 CONJUGATE OPERATORS FOR DYNAMIC DEPENDENCE GRAPHS

In this section we introduce Boolean algebras and other preliminaries (§ 3.1); dynamic dependence graphs (§ 3.2), and the image and preimage functions for a relation and their De Morgan duals (§ 3.3). We then extend these operators to dependence graphs (§ 3.4) via their IO relation, and give algorithms for computing them over a given graph (§ 3.5).

3.1 Preliminaries

We give some notation used in the rest of the paper. X, Y range over finite sets.

Finite sequences. Write $\vec{x}$ to denote a finite sequence of elements $x_1, \dots, x_n$, with ϵ as the empty sequence. Concatenation of two sequences $\vec{x}$ and $\vec{x}'$ is written $\vec{x} ++ \vec{x}'$. We also write $x \cdot \vec{x}$ for cons (prepend) and $\vec{x} \cdot x$ for snoc (append).

Finite maps. Write $\overrightarrow{\{k : x\}}$ to denote a set of pairs $k_1 : x_1, \dots, k_n : x_n$ where keys k_i are pairwise unique.

Disjoint union. Write $X \uplus Y$ to mean $X \cup Y$ where X and Y are disjoint, and also write $x \cdot X$ or $X \cdot x$ to mean $X \uplus \{x\}$. For finite maps ϕ, ψ write $\phi \uplus \psi$ to mean $\phi \cup \psi$ where $\text{dom}(\phi)$ and $\text{dom}(\psi)$ are disjoint.

Relative complement. Write $X \setminus Y$ for the relative complement of Y in X , i.e. the set of elements of X that are not in Y .

Relational converse. Write $\cdot^{-1}$ for the involutive operation that takes a relation $R \subseteq X \times Y$ to its converse $R^{-1} \subseteq Y \times X$ where $(y, x) \in R^{-1} \iff (x, y) \in R$.

3.1.1 Boolean algebras. A *Boolean algebra* (or Boolean lattice) is a 6-tuple $A = (A, \wedge, \vee, \perp, \top, \neg)$ with carrier A , distinguished elements bottom ($\perp$) and top ($\top$), commutative operations meet ($\wedge$) and join ($\vee$) with $\top$ and $\perp$ as respective units, and a unary operation negate ($\neg$) satisfying $x \vee \neg x = \top$ and $x \wedge \neg x = \perp$. The operations $\wedge$ and $\vee$ distribute over each other and induce a partial order $\leq$ over X .

Often we will be considering a specific finite Boolean algebra, namely the powerset $\mathbb{P}(X)$ whose carrier is the set of subsets $X' \subseteq X$ ordered by inclusion $\subseteq$. Bottom is the empty set $\emptyset$, top is the whole set X , meet and join are given by $\cap$ and $\cup$ and negation by relative complement, so that $\neg X' = X \setminus X'$.

3.2 Dynamic Dependence Graphs

We explain how we arrive at specific dependence graphs in § 4, and here just introduce the basic notion. A *dynamic dependence graph* [Agrawal and Horgan 1990] is a directed acyclic graph $G = (V, E)$ with a set V of *vertices* and set $E \subseteq V \times V$ of edges. In the dependence graph for a particular program, vertices $\alpha, \beta \in V$ represents *parts* of values (either supplied to the program as inputs or produced during evaluation) and edges $(\alpha, \beta) \in E$ indicate that, in the evaluation of the program, the partial value associated to β *depends on* the partial value associated to α . (The notion of a “part” of a value will be made precise in § 4.) This diverges somewhat from traditional approaches to dynamic dependence graphs in only considering one type of edge (data dependency) rather than separate data and control dependencies; moreover our edges point in the direction of dependent vertices, whereas in the literature the other direction is somewhat more common.

Definition 3.1 (In-edges and out-edges). For a graph $G = (V, E)$ and vertex $\alpha \in V$, write $\text{in}_{E_G}(\alpha)$ for the in-edges of α in G and $\text{out}_{E_G}(\alpha)$ for its out-edges.

Definition 3.2 (Sources and sinks). For a graph $G = (V, E)$, write $S(G)$ for the *sources* of G (i.e. those vertices with no in-edges) and $T(G)$ for the *sinks* of G (those with no out-edges).

Definition 3.3 (Reachability relation). Define the *reachability relation* for a graph $G = (V, E)$ to be the reflexive transitive closure of E .

Definition 3.4 (Opposite graph). For a graph G define the *opposite graph* $G^{-1} := (V, E^{-1})$ where $\cdot^{-1}$ denotes relational converse.

(Taking the opposite graph has the effect of swapping sources and sinks.)

3.2.1 IO Relation for a Graph. Reachability in G induces a relation just between sources and sinks, which we call the *IO relation* of G .

Definition 3.5 (IO relation). For any dependence graph G with reachability relation R , define the *IO relation* of G to be $R \cap (S(G) \times T(G))$.

The IO relation specifies how specific inputs (sources) are demanded by specific outputs (sinks); thus we will interpret $(x, y) \in R$ as “ x is *demanded by* y ”. Clearly the IO relation of G is the converse of the IO relation of G^{-1} . Since we are in fact interested in the relationship between input and outputs *selections* (namely sets of inputs and outputs), we now consider how such a relation $R \subseteq X \times Y$ lifts to $\mathbb{P}(X)$ and $\mathbb{P}(Y)$ via the *image* and *preimage* functions for R , which we will denote using Δ and ∇ .

3.3 Conjugate Image and Preimage Functions

Definition 3.6 (Image and Preimage Functions for a Relation). For a relation $R \subseteq X \times Y$, define $\Delta_R : \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$ and $\nabla_R : \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$ as:

- (1) $\Delta_R(X') := \{y \in Y \mid \exists x \in X'. (x, y) \in R\}$ (image)
- (2) $\nabla_R(Y') := \{x \in X \mid \exists y \in Y'. (x, y) \in R\}$ (preimage)

Clearly the image function for R is the preimage function for its converse; this is important enough to state as a lemma:

LEMMA 3.7 (IMAGE AS PREIMAGE OF CONVERSE RELATION). $\Delta_R = \nabla_{R^{-1}}$

When thinking of R as an IO relation, the image $\Delta_R(X')$ consists of all outputs that any element of X' is demanded by, and the preimage $\nabla_R(Y')$ consists of all inputs demanded by any element of Y' . If R happens to be a function, the image and preimage functions form a *Galois connection* (Definition 3.12 below); for a proper relation they form a *conjugate pair*, in the sense of Jonsson and Tarski [1951]:

Definition 3.8 (Conjugate Functions). For Boolean algebras A, B , functions $f : A \rightarrow B$ and $g : B \rightarrow A$ form a *conjugate pair* iff:

$$f(x) \wedge y = \perp \iff x \wedge g(y) = \perp$$

(Jonsson and Tarski consider only endofunctions; here we extend the idea of conjugacy to maps between Boolean algebras.)

LEMMA 3.9 (PREIMAGE AND IMAGE OF A RELATION ARE CONJUGATE). *For any relation $R \subseteq X \times Y$ the functions Δ_R and ∇_R are conjugate.*

This should be intuitive enough: for any subsets $X' \subseteq X$ and $Y' \subseteq Y$, if the elements “on the right” to which X' is related are disjoint from Y' , then there are no edges in R from X' to Y' ; and in virtue of that, the elements “on the left” to which Y' is related must also be disjoint from X' .

3.3.1 *Dual Image and Preimage Functions.* If R is an IO relation, then $\Delta_R(X)$ effectively asks “what is X necessary for?” This naturally leads to the question “what is X sufficient for?” In fact, we can use the notion of sufficiency as a test of correctness. If we begin by selecting an output of interest, we would expect the data it demands to also be sufficient to reconstruct it. This leads us to define the following pair of functions, which also form a conjugate pair.

Definition 3.10 (Dual Image and Preimage Functions for a Relation). For any relation $R \subseteq X \times Y$, define $\blacktriangle_R : \mathbb{P}(X) \rightarrow \mathbb{P}(Y)$ and $\blacktriangledown_R : \mathbb{P}(Y) \rightarrow \mathbb{P}(X)$ as the composition:

- (1) $\blacktriangle_R(X') := \{y \in Y \mid \nexists x \in X \setminus X'. (x, y) \in R\}$ (dual image)
- (2) $\blacktriangledown_R(Y') := \{x \in X \mid \nexists y \in Y \setminus Y'. (x, y) \in R\}$ (dual preimage)

This definition is similar to Δ_R except that $\blacktriangle_R$ needs every ancestor to be in the input set, rather than any ancestor to be in the input set. We can, in fact, turn this query on sufficiency into Δ_R , using the De Morgan Dual.

Definition 3.11 (De Morgan Dual). Suppose a function $f : A \rightarrow B$ between Boolean algebras with $\neg_A$ and $\neg_B$ the negation operators of A and B . Define the *De Morgan Dual* $f^\circ : A \rightarrow B$ of f as:

$$f^\circ := \neg_B \circ f \circ \neg_A$$

Definition 3.12 (Galois connection). Suppose A, B are partial orders. Then monotone functions $f : A \rightarrow B$ and $g : B \rightarrow A$ form a *Galois connection* iff

$$f(x) \leq y \iff x \leq g(y)$$

PROPOSITION 3.13 (CORRESPONDENCE BETWEEN CONJUGATES AND GALOIS CONNECTIONS). *Suppose functions $f : A \rightarrow B$ and $g : B \rightarrow A$ between Boolean algebras. The following statements are equivalent:*

- (1) f and g form a conjugate pair
- (2) f and g° form a Galois connection

LEMMA 3.14 (DUALITY OF IMAGE AND PREIMAGE FUNCTIONS).

- (1) $\Delta_R^\circ = \blacktriangle_R$
- (2) $\nabla_R^\circ = \blacktriangledown_R$

3.4 ∇ and Δ over Dependence Graphs

We now extend the notion of ∇ and Δ to a dependence graph G via its IO relation. The set of outputs *demanded* by some inputs X is simply the subset of the outputs of G reachable from X , i.e. the intersection of $\Delta_R(X)$ with $T(G)$. Conversely, the set of inputs that some outputs Y *demands* is simply the subset of the inputs of G reachable from Y by traversing the graph in the opposite direction, i.e. the intersection of ∇_R with $S(G)$.

Definition 3.15 (∇ and Δ for a dependence graph). For a graph G with IO relation R , define:

- (1) $\Delta_G := \Delta_R : \mathbb{P}(S(G)) \rightarrow \mathbb{P}(T(G))$ (demanded by)
- (2) $\nabla_G := \nabla_R : \mathbb{P}(T(G)) \rightarrow \mathbb{P}(S(G))$ (demands)

3.5 Computing ∇_G and Δ_G and their duals

We now show how to compute Δ_G and ∇_G and their duals $\blacktriangle_G$ and $\blacktriangledown_G$ for a dependence graph G . One advantage of graphs over the trace-based approach of [Perera et al. \[2022\]](#) is that a single algorithm can be readily adapted to compute all 4 operators; given a procedure for Δ_G , for example, we can compute its conjugate ∇_G as $\Delta_{G^{-1}}$ (by Lemma 3.7) and its dual $\blacktriangle_G$ by pre- and post-composing with negation (Definition 3.11).

For efficiency, however, we give a direct procedure both for Δ_G (§ 3.5.1) and $\blacktriangle_G$ (§ 3.5.2), allowing us to compute the other two operators via the opposite graph; this is in some situations more efficient than relying on the De Morgan dual, which can cause more graph to be traversed than would be otherwise. Both procedures factor through an auxiliary operation that computes a “slice” of the original graph, i.e. a contiguous subgraph. While it is technically possible to compute the desired image/preimage of the IO relation without creating this intermediate graph, we anticipate many use cases which will make use of the graph slice; these are discussed in more detail in § 7. Our implementation makes it easy to flip between G and G^{-1} so we freely make use of both in the algorithms to access out-neighbours and in-neighbours.

An alternative presentation of graphs in terms of adjacency maps is useful notationally.

Definition 3.16 (Graph as adjacency map). A graph $G = (V, E)$ can also be specified as an *adjacency map* $G : V \rightarrow \mathbb{P}(V)$ from vertices to sets of out neighbours, where

$$G(\alpha) = V' \iff \alpha \in V \text{ and } \forall \beta \in V'. (\alpha, \beta) \in E$$

3.5.1 Direct algorithm for Δ_G (demanded by).

Definition 3.17 (demBy_G). Figure 5a defines the family of relations demBy_G for dependence graph G .

For a dependence graph G and inputs $X \subseteq S(G)$, the judgement $X \text{ demBy}_G Y$ says that X is demanded by $Y \subseteq T(G)$. It is possible to read an algorithm directly from the inductive definition, treating it as a function with V and index G as inputs.¹ The auxiliary operation $G \text{ demBy}_G V_G$ maintains a set of vertices still to process and builds the graph slice $G' \subseteq G$ consisting of all vertices reachable from the initial set and all edges between them. If X is empty (done) the graph slice G' is complete and can be returned. Otherwise we select an arbitrary vertex α from X . If α is already in G' (skip), there is nothing to do at this step and we continue with the remaining vertices. Otherwise, we add the out-star at α to G' and the out-neighbours to the pending vertices (extend). When demByV is done, demBy extracts the output set from the graph slice by intersecting with $T(G)$.

PROPOSITION 3.18 (demBy COMPUTES Δ). *For any dependence graph G , any $X \subseteq S(G)$ and any $Y \subseteq T(G)$ we have*

$$X \text{ demBy}_G Y \iff \Delta_G(X) = Y$$

3.5.2 Direct algorithm for $\blacktriangle_G$ (suffices).

Definition 3.19 (suff_G). Figure 5b defines the family of relations suff_G for dependence graph G .

For a dependence graph G , the algorithm suff also delegates to an auxiliary procedure suffE which builds a graph slice G' , similar to demands except that the direction of the slice is inverted, i.e. $G' \subseteq G^{-1}$. The judgement suffE takes the graph slice under construction G' , a “pending” subgraph $H \subseteq G$ with $S(H) \subseteq T(G')$, and the current set of edges under consideration E . For efficiency, the algorithm traverses each edge in the graph at most once.

Figure 6 shows an example run of the algorithm. Green vertices have been included into G' because all of their in-edges in G are out-edges in G' ; orange vertices are “pending”, with at least one in-edge from G as an out-edge in H ; blue vertices are unvisited. Dashed edges indicate the current edge set E . At each step we non-deterministically choose an edge (α, β) from E and add

¹Any choice between skip and extend is made deterministic by the disjointness of G and H in extend implying the opposite side condition to skip ($\alpha \notin V(G)$), and with termination implied by eventual saturation of the first argument by all of the edges in the indexing graph leading from the vertices in second argument, after which a finite sequence of skip can be applied to reach the base case of done.

$X \text{ demBy}_G Y$

$$\frac{(\emptyset, \emptyset), X \text{ demBy}_G G'}{X \text{ demBy}_G V(G') \cap T(G)} X \subseteq S(G)$$

$G', X \text{ demBy}_G G''$

done

$$\frac{}{G', \emptyset \text{ demBy}_G G'} G' \subseteq G$$

skip

$$\frac{G', X \text{ demBy}_G G''}{G', \alpha \cdot X \text{ demBy}_G G''} \alpha \in V(G)$$

extend

$$\frac{G' \uplus H, Y \uplus X \text{ demBy}_{G \uplus H} G''}{G', \alpha \cdot X \text{ demBy}_{G \uplus H} G''} H = \{\alpha : Y\}$$

(a) demBy algorithm

$X \text{ suff}_G U$

$$\frac{(X, \emptyset), \emptyset, \{(\alpha, \beta) \in \text{outE}_G(\alpha') \mid \alpha' \in X\} \text{ suffE}_G G', H}{X \text{ suff}_G V(G') \cap S(G)}$$

$G', H, E \text{ suffE}_G G'', H'$

done

$$\frac{}{G', H, \emptyset \text{ suffE}_G G', H'} G' \subseteq G^{-1}$$

pending

$$\frac{H'(\beta) \subset G^{-1}(\beta) \quad G', H', E \text{ suffE}_G G'', H''}{G', H, (\alpha, \beta) \cdot E \text{ suffE}_G G'', H''} H' = H \cup \{\beta : \{\alpha\}\}$$

extend

$$\frac{H'(\beta) = G^{-1}(\beta) \quad G' \uplus \{\beta : H'(\beta)\}, H \setminus \{\beta\}, \text{outE}_G(\beta) \uplus E \text{ suffE}_G G'', H''}{G', H, (\alpha, \beta) \cdot E \text{ suffE}_G G'', H''} H' = H \cup \{\beta : \{\alpha\}\}$$

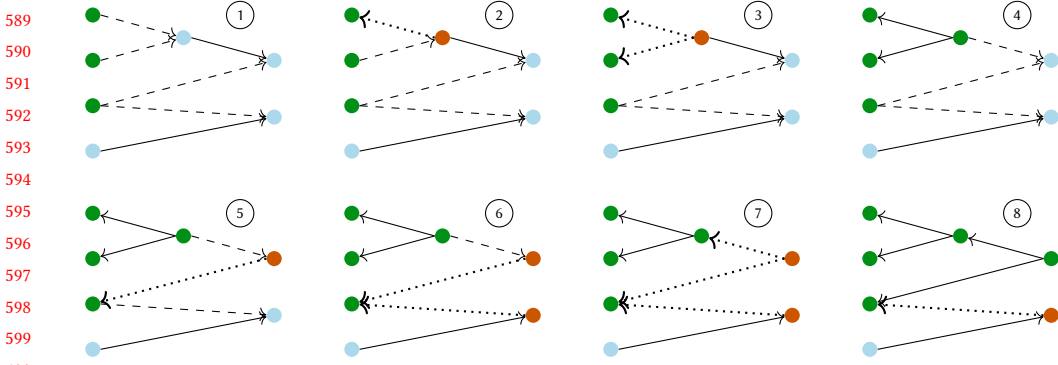
(b) suff algorithm

Fig. 5. Dual analyses over a dependence graph G

the opposite edge (β, α) to H (dotted edges in steps 2 and 3). If the out-neighbours of β in H are now equal to its in-neighbours in G (as in step 3), we move that out-star from H to G and add the in-edges of β in G to E (suffE-extend, step 4). Otherwise, there is nothing more to do at this step and we continue with the remaining edges in E (suffE-pending). The algorithm terminates when E is empty (suffE-done); any vertices still pending in H (as in step 8) are vertices had only partially sufficient inputs.

PROPOSITION 3.20 (suff COMPUTES ▲). *For any dependence graph G , any $X \subseteq S(G)$ and any $Y \subseteq T(G)$ we have:*

$$X \text{ suff}_G Y \iff \blacktriangle(X) = Y$$

Fig. 6. Run of suff with G and H superimposed on G_0

3.5.3 *Derived algorithms.* Propositions 3.18 and 3.20 justify treating demBy_G and suff_G as functions. Using Lemma 3.14 we can define direct algorithms for ∇_G and $\blacktriangledown_G$ from demBy_G and suff_G by simply flipping the graph; using Lemma 3.14 and Definition 3.11 we can also define alternative algorithms for all 4 operators using the De Morgan dual construction. All algorithms are summarised in Figure 7.

Abstract operator	Direct algorithm	Alternative algorithm
Δ_G	demBy_G	suff_G°
∇_G	$\text{demBy}_{G^{-1}}$	$\text{suff}_{G^{-1}}^\circ$
$\blacktriangle_G$	suff_G	demBy_G°
$\blacktriangledown_G$	$\text{suff}_{G^{-1}}$	$\text{demBy}_{G^{-1}}^\circ$

Fig. 7. Direct and alternative (De Morgan Dual) algorithms for all 4 operators

In § 5 we contrast the performance of the direct and De Morgan dual implementations of Δ_G , which is an essential component of the related inputs and related outputs operators introduced in § 2.

3.6 $\nabla\Delta$ and $\Delta\nabla$ over dependence graphs

Now we can provide a formal account of the notions of related inputs and related outputs.

Definition 3.21 (Related Inputs and Outputs). For a dependence graph G define:

- (1) $\nabla\Delta_G := \nabla_G \circ \Delta_G$ (related inputs)
- (2) $\Delta\nabla_G := \Delta_G \circ \nabla_G$ (related outputs)

Intuitively, related outputs and related inputs are relations of *cognacy* (common ancestry) in G and G^{-1} respectively. For a set of inputs $X \subseteq S(G)$, related inputs asks “what other inputs are demanded by the outputs which demand X ?”, and first finds all outputs $\Delta_G(X)$ that our inputs are demanded by, and then computes the inputs that those outputs demand, i.e. $\nabla_G(\Delta_G(X'))$. For a set of outputs $Y \subseteq T(G)$, related outputs asks “what other outputs demand the inputs that Y demands”, and first finds all inputs $\nabla_G(Y')$ that our outputs demand, and then computes the outputs that those inputs $\nabla_G(Y)$ demand, i.e. $\Delta_G(\nabla_G(Y))$.

Neither $\nabla\Delta_G$ nor $\Delta\nabla_G$ is necessarily inflationary, as illustrated in Figure 8. In Figure 8a, the uppermost vertex on the left is part of the initial input selection X' , but is not preserved by the $\nabla_G \circ \Delta_G$ round-trip because it is not demanded by any output. In Figure 8b, the uppermost vertex

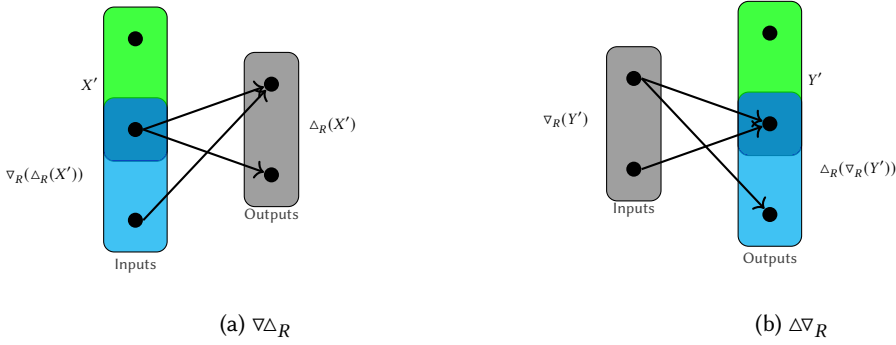


Fig. 8. In general neither $\nabla\Delta_R$ nor $\Delta\nabla_R$ is inflationary

on the right is part of the initial output selection Y' , but is not preserved by the $\Delta_G \circ \nabla_G$ round-trip because it does not demand any input.

If we constrain things so that there are no inputs which are unused, then ∇ will preserve $\perp$, i.e. $\nabla(\emptyset) = \emptyset$, and $\Delta\nabla_G$ will preserve the original output selection. Similarly, if we constrain things so that all outputs demand some input, then Δ will preserve $\perp$, and $\nabla\Delta_G$ will preserve the original input selection:

LEMMA 3.22.

- (1) If $\Delta_G(\emptyset) = \emptyset$, then for all X we have $\nabla\Delta_G(X) \geq X$.
- (2) If $\nabla_G(\emptyset) = \emptyset$, then for all Y we have $\Delta\nabla_G(Y) \geq Y$.

As an easy corollary, if ∇_G and Δ_G both preserve $\emptyset$ then in fact they form an antitone Galois connection. These observations have practical implications: for example it is sometimes convenient to consider a filtered view of the data where by default the user only sees data which is actually used somewhere, and in such a setup a related inputs query will never cause any selected data to become unselected.

4 A CORE LANGUAGE WITH GRAPHICAL SYNTAX AND SEMANTICS

In § 3, we defined the abstract setting of dependence graphs and conjugate operators *demands* (∇) and *demanded by* (Δ) over such graphs. We now implement that graphical abstraction for [OurLanguage], an untyped, pure functional programming language. This core calculus we go on to describe is what underlies the surface-language programs such as Figure 4, used to generate and analyse the scatter plots from earlier. We first introduce the syntax of our core language constructs (§ 4.1). Then we define a big-step operational semantics that simultaneously (i) evaluates a term to a value and (ii) constructs an accompanying dependence graph for the evaluation of that term (§ 4.2); we also address how we deal with pattern-matching and mutual recursion.

4.1 Syntax

4.1.1 Terms. Figure 9 defines the expressions e of the language, which include variables x, y, f ; integer constants n, m ; let-bindings $\text{let } x = e \text{ in } e'$; record construction $\{\overline{x} : \overline{e}\}$; and record projection $e.x$. We also have constructor expressions $c(\overline{e})$, where c ranges over data constructors and is fully saturated by arguments $\overline{e}$; we write $\Sigma(c) = n$ to denote that c takes $n \in \mathbb{N}$ arguments. Application comes in two forms: standard application $e e'$, and *foreign application* $f(\overline{e})$ for calling foreign functions which are explained in § 4.1.2. The last two terms define anonymous functions

Expression			Eliminator		
$e ::= x$	variable		$\sigma ::= x \mapsto \kappa$	variable	
n	integer		$\{\vec{x}\} \mapsto \kappa$	record	
$\text{let } x = e \text{ in } e'$	let		$\{c \mapsto \vec{\kappa}\}$	constructor	
$\{\vec{x} : \vec{e}\}$	record		<i>Value</i>		
$e.x$	record projection		$v ::= v_\alpha$		
$c(\vec{e})$	constructor		<i>Raw value</i>		
$e e'$	application		$v ::= n$	integer	
$f(\vec{e})$	foreign application		$\{\vec{x} : \vec{d}\}$	record	
$\lambda \sigma$	function		$c(\vec{v})$	constructor	
$\text{let } \vec{x} : \vec{\sigma} \text{ in } e$	recursive let		$\text{cl}(\gamma, \rho, \sigma)$	closure	
<i>Continuation</i>			<i>Environment</i>		
$\kappa ::= e$	expression		$\gamma ::= \{\vec{x} : \vec{d}\}$		
σ	eliminator		<i>Recursive definitions</i>		
			$\rho ::= \{\vec{x} : \vec{\sigma}\}$		

Fig. 9. Syntax of core language, including annotated values

$\lambda \sigma$ and (mutually) recursive let-bindings $\text{let } \vec{x} : \vec{\sigma} \text{ in } e$, where σ is a pattern-matching construct called an *eliminator* detailed further in § 4.1.3.

4.1.2 Foreign functions. The language is parameterised by a finite map $\Phi(f) = n$ from variables f to the arities $n \in \mathbb{N}$ of the foreign function they represent (analogous to $\Sigma(c) = n$ for constructors). For the graph semantics that will follow, every foreign function f is required to provide an interpretation $\hat{f}$ that computes both f 's result and a suitable dependence graph.

Foreign functions in the core language are not first-class and calls $f(\vec{e})$ are saturated, i.e. $|\vec{e}| = \Phi(f)$. In our implementation, which also provides a surface language which desugars into the core, first-class foreign functions are supported via a top-level environment which maps every f to the closure $\text{cl}(\emptyset, \epsilon, x_1 \mapsto \dots \mapsto x_{\Phi(f)} \mapsto f(\vec{x}))$ where $|\vec{x}| = \Phi(f)$.

4.1.3 Continuations and eliminators. A *continuation* κ describes how an execution proceeds after a value is pattern-matched, and is either a term e or an *eliminator* σ . Eliminators are trie-like objects [Hinze 2000] that specify how this pattern-matching is done and the subsequent continuation κ to be executed under additional variable bindings. (These are what the piecewise function definitions of our surface-language, e.g. `plot` in Figure 4, desugar into; Peyton Jones et al. [2022] present a similar idea.) A variable eliminator $x \mapsto \kappa$ says how to match any value (as variable x) and continue as κ . A record eliminator $\{\vec{x}\} \mapsto \kappa$ says how to match a record with fields $\vec{x}$, and provides a continuation κ for matching the values of those fields. Lastly, the eliminator $\{c \mapsto \vec{\kappa}\}$ provides a branch $c \mapsto \kappa$ for each constructor c in $\vec{c}$, where each κ may specify how the arguments to c will be matched. (We assume that any constructors matched by an eliminator all belong to the same data type, but this is not enforced in the core language.)

4.1.4 Values and environments. We define addressed *values* v mutually inductively with *raw values* ν , a raw value does not have an associated address. These addresses, explained in § 4.1.5, will (later) form the vertices of the graphs that we build in the graph semantics.

Raw values ν include integers n, m ; records $\{\vec{x} \mapsto \vec{v}\}$; and constructor values $c(\vec{v})$. We also define function closures as $\text{cl}(\gamma, \rho, \sigma)$ where σ is the function (eliminator), γ is its environment, and to

$$\begin{array}{c}
\boxed{\vec{v}, \kappa \rightsquigarrow \gamma, e, V} \\
\\
\begin{array}{cc}
\rightsquigarrow\text{-done} & \rightsquigarrow\text{-var} \\
\hline
\epsilon, e \rightsquigarrow \emptyset, e, \emptyset & \frac{\vec{v}, \kappa \rightsquigarrow \gamma, e, V}{v \cdot \vec{v}, x \mapsto \kappa \rightsquigarrow \gamma \cdot (x : v), e, V} \\
\\
\rightsquigarrow\text{-record} & \rightsquigarrow\text{-constr} \\
\frac{\{\vec{y} : \vec{u}\} \subseteq \{\vec{x} : \vec{v}\} \quad \vec{u} ++ \vec{v}', \kappa \rightsquigarrow \gamma, e, V}{\{\vec{x} : \vec{v}\}_\alpha \cdot \vec{v}', \{\vec{y}\} \mapsto \kappa \rightsquigarrow \gamma, e, \alpha \cdot V} & \frac{\vec{v} ++ \vec{v}', \kappa \rightsquigarrow \gamma, e, V}{c(\vec{v})_\alpha \cdot \vec{v}', (c \mapsto \kappa) \cdot \{\vec{c} \mapsto \vec{k}\} \rightsquigarrow \gamma, e, \alpha \cdot V} \Sigma(c) = |\vec{v}|
\end{array}
\end{array}$$

Fig. 10. Pattern-matching

support mutual recursion, ρ is the (possibly empty) environment of functions (eliminators) with which σ was mutually defined. An environment is defined as a finite map.

4.1.5 Addresses for partial values. The addresses α that annotate values act as identifiers, uniquely corresponding those values to specific vertices in a dependence graph. There are many sensible ways of assigning addresses (our particular implementation uses an integer counter to generate new ones) as long as some freshness condition is satisfied – ensuring that there are no unwanted address collisions in a given value i.e. graph. For simplicity, we elide this side-condition in the graph semantics (next) and assume it is met whenever a new address α is introduced.

4.2 Evaluation and Graph Construction

4.2.1 Pattern matching. Figure 10 defines the pattern-matching judgement $\vec{v}, \kappa \rightsquigarrow \gamma, e, V$. Rather than matching a single value v against an eliminator σ , the judgement matches a “stack” of values $\vec{v}$ against a continuation κ . It then returns the selected branch e , an environment γ providing bindings for the free variables in e , and a set V of vertices found in the matched portions of $\vec{v}$.

The $\rightsquigarrow\text{-done}$ rule expresses exactly when using a *term* e as the continuation may succeed: when the stack of values to be matched is empty, and so e is treated as the selected term to be executed. The other rules require an *eliminator* σ as the continuation – using this to match the head of a non-empty stack $v \cdot \vec{v}$, unpack any of its relevant subvalues, and then push them onto the tail $\vec{v}$ to be recursively matched by the continuation selected in σ .

A variable eliminator $x \mapsto \kappa$ matches any value v and returns no vertices, reflecting the fact that no part of v would need to be consumed to select κ as the continuation. The eliminator $\{\vec{y}\} \mapsto \kappa$ only matches with a record $\{\vec{x} : \vec{v}\}_\alpha$ if the variables in $\vec{y}$ are also fields in $\vec{x}$, returning the vertex α associated with the record; this condition is specified by the premise $\{\vec{y} : \vec{u}\} \subseteq \{\vec{x} : \vec{v}\}$, which simultaneously projects out the corresponding values of $\vec{u}$ from $\vec{v}$. Lastly, an eliminator with a branch $(c \mapsto \kappa)$ only matches a value $c(\vec{v})_\alpha$ constructed by c , and returns the vertex α associated with that particular instance of c .

4.2.2 Evaluation. Figure 11 defines a big-step evaluation relation $\gamma, e, V \Rightarrow v, G$ for the core language, stating that term e , under an environment γ and vertex set V , evaluates to a value v plus dependence graph G . The input set V corresponds to the inputs consumed by the current active function call from which e is being evaluated, and can be thought of as the “ambient” set of dependencies; this is initially empty for a top-level term, and may grow whenever a function is applied to an argument and pattern-matching occurs (i.e. in the $\Rightarrow\text{-app}$ rule, which we explain later). The output graph G is built by specifying new graph fragments and then combining them; for convenience, we write $\{V \leftarrow \alpha\}$ as shorthand for the star graph $(\{\alpha\} \cup V, V \times \{\alpha\})$ indicating that vertex α depends on all vertices in V .

$$\begin{array}{c}
\boxed{\gamma, e, V \Rightarrow v, G} \\
\Rightarrow\text{-var} \qquad \qquad \qquad \Rightarrow\text{-int} \qquad \qquad \qquad \Rightarrow\text{-function} \\
\frac{}{\gamma \cdot (x : v), x, V \Rightarrow v, \emptyset} \qquad \frac{}{\gamma, n, V \Rightarrow n_\alpha, \{V \leftarrow \alpha\}} \qquad \frac{}{\gamma, \lambda\sigma, V \Rightarrow \text{cl}(\gamma, \emptyset, \sigma)_\alpha, \{V \leftarrow \alpha\}} \\
\Rightarrow\text{-record} \qquad \qquad \qquad \Rightarrow\text{-constr} \\
\frac{\gamma, \vec{e}, V \Rightarrow \vec{v}, G}{\gamma, \{\vec{x} : \vec{e}\}, V \Rightarrow \{\vec{x} : \vec{v}\}_\alpha, G \uplus \{V \leftarrow \alpha\}} \qquad \frac{\gamma, \vec{e}, V \Rightarrow \vec{v}, G}{\gamma, c(\vec{e}), V \Rightarrow c(\vec{v})_\alpha, G \uplus \{V \leftarrow \alpha\}} \Sigma(c) = |\vec{e}| \\
\Rightarrow\text{-record-project} \qquad \qquad \qquad \Rightarrow\text{-foreign-app} \\
\frac{\gamma, e, V \Rightarrow \{\vec{x} : \vec{v} \cdot (y : u)\}_\alpha, G}{\gamma, e.y, V \Rightarrow u, G} \qquad \frac{\gamma, \vec{e}, V \Rightarrow \vec{v}, G \quad \hat{f}(\vec{v}) = (u, G')}{\gamma, f(\vec{e}), V \Rightarrow u, G \uplus G'} \Phi(f) = |\vec{e}| \\
\Rightarrow\text{-let} \qquad \qquad \qquad \Rightarrow\text{-let-rec} \\
\frac{\gamma, e, V \Rightarrow v, G \quad \gamma \cdot (x : v), e', V \Rightarrow v', G'}{\gamma, \text{let } x = e \text{ in } e', V \Rightarrow v', G \uplus G'} \qquad \frac{\gamma, \{\vec{x} : \vec{e}\}, V \mapsto \gamma', G \quad \gamma ++ \gamma', e, V \Rightarrow v, G'}{\gamma, \text{let } \vec{x} : \vec{e} \text{ in } e, V \Rightarrow v, G \uplus G'} \\
\Rightarrow\text{-app} \\
\frac{\gamma, e, V \Rightarrow \text{cl}(\gamma_1, \rho, \sigma)_\alpha, G_1 \quad \gamma_1, \rho, \{\alpha\} \mapsto \gamma_2, G_2 \quad \gamma, e', V \Rightarrow v', G_3 \quad v', \sigma \rightsquigarrow \gamma_3, e'', V' \quad \gamma_1 ++ \gamma_2 ++ \gamma_3, e'', V' \uplus \{\alpha\} \Rightarrow u, G_4}{\gamma, e e', V \Rightarrow u, \uplus\{G_1, \dots, G_4\}} \\
\boxed{\gamma, \vec{e}, V \Rightarrow \vec{v}, G} \\
\Rightarrow\text{-seq} \\
\frac{\gamma, e_i, V \Rightarrow v_i, G_i \quad (\forall i \leq |\vec{e}|)}{\gamma, \vec{e}, V \Rightarrow \vec{v}, \uplus\vec{G}}
\end{array}$$

Fig. 11. Operational semantics: evaluation and graph construction

$$\boxed{\gamma, \rho, V \mapsto \gamma', G} \qquad \text{with } \text{dom}(\gamma') = \text{dom}(\rho)$$

$$\frac{v_i = \text{cl}(\gamma, \rho, \rho(x_i))_{\alpha_i} \quad G_i = \{\alpha_i : V\} \quad (\forall i \leq |\vec{x}|)}{\gamma, \rho, V \mapsto \{\vec{x} : \vec{v}\}, \uplus\vec{G}}$$

Fig. 12. “Closing over” recursive definitions

The $\Rightarrow\text{-var}$ rule for evaluating variables is standard, and the returned dependence graph is empty; that is, no new vertices are constructed as a result of simply looking up values. Most of the other evaluation rules can generally be understood in terms of two recurring patterns.

First, for any introduction rule that creates a new (partial) value, there is no choice but to allocate that value with a new vertex (address) which depends on ambient vertices V . These cases include $\Rightarrow\text{-int}$, $\Rightarrow\text{-function}$, $\Rightarrow\text{-record}$, and $\Rightarrow\text{-constr}$. For example, $\Rightarrow\text{-int}$ evaluates an integer n to its value form n_α assigned a fresh vertex α ; the resulting dependence graph is then $\{\alpha : V\}$, denoting that evaluating to n_α depended on any inputs that were found in V . Likewise, $\Rightarrow\text{-function}$ evaluates an anonymous function $\lambda\sigma$ by constructing the closure $\text{cl}(\gamma, \emptyset, \sigma)_\alpha$ annotated by a new vertex α ; here, the closure captures the current environment γ and uses $\emptyset$ as the set of mutually recursive definitions. The “exception” to this pattern is $\Rightarrow\text{-record-project}$ for $e.y$, which evaluates a record e to $\{\vec{x} : \vec{v} \cdot (y : u)\}_\alpha$ before returning the value u associated to field y ; while this indeed introduces a

record value allocated with a new vertex α , the record as a whole goes unused, and so α is omitted in the final dependence graph.

The second recurring pattern is that, for any rule which involves evaluating subterms (of a term), the corresponding subgraphs that are constructed must be disjoint – reflecting that there is no overlap between their vertex addresses – and their union is the dependence graph of the overall term. This general principle is captured by the auxiliary evaluation relation $\gamma, \vec{e}, V \Rightarrow \vec{v}, G$ (bottom of Figure 11), which is for evaluating a sequence of terms $\vec{e}$. Its single rule evaluates each term $e_i \in \vec{e}$ to a value $v_i \in \vec{v}$ and graph $G_i \in \vec{G}$, and builds the dependence graph $\biguplus \vec{G}$. We make use of this relation in the rules $\Rightarrow$ -record for $\{\bar{x} : \vec{e}\}$, $\Rightarrow$ -constr for $c(\vec{e})$, and $\Rightarrow$ -foreign-app for $f(\vec{e})$, which all involve evaluating a sequence $\vec{e}$ of subterms; in the last rule, $\hat{f}$ is the foreign implementation that provides a way to evaluate f to a graph and value (as required in § 4.1.2). Of course, this pattern applies to subterms in general and not just those expressed as a sequence $\vec{e}$. For example, the $\Rightarrow$ -let rule for $\text{let } x = e \text{ in } e'$ (which otherwise behaves as standard) simply unions the graphs of subterms e and e' , and similarly for $\Rightarrow$ -let-rec and $\Rightarrow$ -app.

To fully explain the last two rules, $\Rightarrow$ -let-rec and $\Rightarrow$ -app, we first detail how our language deals with *mutual recursion*. For this, Figure 12 defines the auxiliary relation $\gamma, \rho, V \mapsto \gamma', G$, which takes an environment ρ of recursive definitions and evaluates it to a value environment γ' of closures, while also building a dependence graph G . This works by turning each function definition $\rho(x_i) \in \rho$ into a newly allocated closure $(\gamma, \rho, \rho(x_i))_{\alpha_i}$ as a fresh vertex α_i , capturing ρ and a copy of the original environment γ , where each closure depends on ambient inputs V (like in evaluation). The final dependence graph is the combination of each closure's subgraph (also like in evaluation).

Returning to Figure 11, rule $\Rightarrow$ -let-rec for $\text{let } \bar{x} : \vec{\sigma} \text{ in } e$ is then the same as regular let-bindings, except using $\mapsto$ to evaluate recursive definitions $\bar{x} : \vec{\sigma}$. The final rule, $\Rightarrow$ -app for $e e'$, is especially important to the graph semantics, as it is during application that we grow the ambient inputs V that a given evaluation step depends on. Here, we compute the closure $\text{cl}(\gamma_1, \rho, \sigma)_{\alpha}$ of function e , and use its eliminator σ to pattern-match against the value v' of argument e' . What is returned includes (i) the *selected branch* e'' and (ii) the vertex set V' , where V' represents the values in v' consumed by pattern-matching in order to lead to e'' . The last premise which executes the branch then specifies its ambient dependencies as $V' \uplus \{\alpha\}$; that is, the (closure of the) function that was applied, α , and the (consumed values of the) argument it was applied to, V' .

4.2.3 Surface Language. Our implementation provides a surface language which desugars into the core language presented above; in particular the surface language supports piecewise function definitions (which desugar into eliminators) and conventional list comprehensions. The core and surface languages also both support matrices, using a comprehension-like syntax for matrix initialisation. The example in Figure 13 illustrates both: line 2 contains a list comprehension, and lines 4-7 and line 28 contain matrix comprehensions. In a matrix comprehension, the variables immediately to the right of the $|$ bind the indices of each matrix element and the initialiser expression to the left of the $|$ may then refer to these variables in order to specify the initial value of that element.

5 EVALUATION

We now present a critical evaluation of our approach. We start by considering its qualitative advantage in terms of conceptual simplicity and ease of implementation with respect to the state of the art in the field, [Perera et al. \[2022\]](#). Then, we provide an experimental comparison of the performance of our implementation and that of [Perera et al. \[2022\]](#).

5.1 Qualitative Evaluation of Our Approach

Here, we consider the relative implementation burden of the current graph approach compared to the trace approach in [Perera et al. \[2022\]](#). As explained below, we hypothesise that our approach is easier to implement and maintain compared to the trace approach.

Firstly, a key benefit of the graph approach is that it avoids the need to implement a backwards evaluation function. By contrast, in [Perera et al. \[2022\]](#), a program produces an execution trace that is rewound in order to perform a backwards analysis such as ∇ . Consequently, the developer has to implement a separate backwards evaluation semantics using the execution trace, which propagates a selected output back to the program inputs. This approach is burdensome since it requires the forwards and backwards evaluation functions to “agree” with respect to the dependency analysis. Conversely, in our approach, once the dependence graph has been computed, forward evaluation is used to compute both ∇ and Δ over the graph.

Secondly, our algorithm for computing ∇ and Δ is language-agnostic (as described in § 3), meaning that it is not affected by changes in the semantics of the language. Thus, as opposed to [Perera et al. \[2022\]](#), we only need to implement one language-specific algorithm.

5.2 Experimental Evaluation

In order to test the performance of our approach, we implemented our language from § 4 on top of PureScript, which we used to write a selection of non-trivial examples that reflect realistic targets of dependency analyses (§ 5.2.1). We used PureScript since this is the implementation language used by [Perera et al. \[2022\]](#) in their open-source implementation of the trace-based approach, thus enabling a more fair like-for-like comparison to our work here. PureScript is also a natural implementation language for this work as our focus on visualizations naturally lends itself to building web-based interfaces. We then benchmarked the implementations on an Intel Core i7-10850H 2.70GHz with 16Gb of RAM, using Google Chrome 121.0.6261.69 and JavaScript runtime V8 12.2.281.16. We instrumented both systems in order to extract performance numbers.

In an interactive system, performance plays a critical role in a user’s experience. Thus, we compare the performance of our graph-based approach with the trace-based [[Perera et al. 2022](#)], refined to the following three research questions:

- RQ1: What is the overhead of the graph construction versus the trace construction?
- RQ2: How does the performance of graph-based *demands* compare against the performance of trace-based *demands*?
- RQ3: How does the performance of the various implementations of *demanded by* (i.e. trace-based, graph-based using demands, and graph-based using the dual of suff) compare?

5.2.1 Choice of examples. We tested our system on a range of benchmark examples implemented in [OurLanguage] and in the language of [Perera et al. \[2022\]](#). The benchmark programs represent a range of components a data scientist may find useful in an exploratory data analysis scenario. First, we considered matrix convolution (**edgeDetect**, **emboss**, **gaussian**, 31 Lines of Code (LoC) each). Given that different filter/kernel matrices create subtly different dependency structures, we implemented a few different such kernels. Second, we include an example of a more involved algorithm with a deeper sequential dependency structure: an implementation of dynamic time warping [[Berndt and Clifford 1994](#)] known from time series analysis (**compute-dtw**, 37 LoC). We provide a view of this program in Figure 13. Third, we developed a prototype of a graphics library that demonstrates the performance on bespoke visualization code (**grouped-bar-chart**, 140 LoC, **line-chart**, 143 LoC, **stacked-bar-chart**, 136 LoC). Finally, we test two linked examples (**stacked-bar-scatter-plot** (corresponding to Figure 2 from § 2), 26 LoC, and **bar-chart-line-chart**, 38 LoC). The rest of the benchmark code is listed in the appendix.

```

932 1 let nextIndices n m window =
933 2   [(i, j) | i ← [1 .. n], j ← [max 1 (i - window) .. min m (i + window)]];
934 3 let costMatrixInit rows cols window =
935 4   [(let initV = if ((n == 1) `and` (m == 1)) `or` ((abs n m <= window)
936 5     `and` not ((n == 1) `or` (m == 1)))
937 6     then FNum 0 else Infity
938 7     in initV | (n, m) in (rows, cols)]];
939 8 let minAndPrev (i, j) i1 j1 i2 j2 =
940 9   let minim = minimal [i1, j1, i2, j2] in
941 10    if minim `eq` i1 then ((i, j + 1), minim)
942 11    else if minim `eq` j1 then ((i + 1, j), minim) else ((i, j), minim);
943 12 let extractPath indmatrix (n, m) accum =
944 13   if (n == 1) `and` (m == 1)
945 14   then accum
946 15   else extractPath indmatrix (indmatrix!(n, m)) ((n - 1, m - 1) : accum);
947 16 let localMinUpdate seq1 seq2 cost (costmatrix, indmatrix) (i, j) =
948 17   let (iEntr, jEntr) = (nth (i - 1) seq1, nth (j - 1) seq2);
949 18   dist = cost iEntr jEntr;
950 19   (ip, jp) = (i + 1, j + 1);
951 20   (i1, j1) = (costmatrix!(i, jp), costmatrix!(ip, j));
952 21   (i2, j2) = (costmatrix!(i, j), costmatrix!(ip, jp));
953 22   (prev, FNum minim) = minAndPrev (i, j) i1 j1 i2 j2;
954 23   newVal = FNum (dist + minim)
955 24   in (matrixUpdate costmatrix (ip, jp) newVal, matrixUpdate indmatrix (ip, jp) prev);
956 25 let computedDTW seq1 seq2 cost window =
957 26   let (n, m) = (length seq1, length seq2);
958 27   initD = costMatrixInit (n + 1) (m + 1) window;
959 28   initI = [(0 | (i, j) in (n + 1, m + 1))];
960 29   indexing = nextIndices n m window;
961 30   (finished, indices) = foldl (localMinUpdate seq1 seq2 cost) (initD, initI) indexing
962 31   in (finished, extractPath indices (n + 1, m + 1) Nil);
963 32 let d x y = (x - y) * (x - y) in
964 33 let seq1 = [3, 1, 2, 2, 1];
965 34 let seq2 = [2, 0, 0, 3, 3, 1, 0];
966 35 window = 2;
967 36 (costs, matched) = computedDTW seq1 seq2 d window
968 37 in matched

```

Fig. 13. Source code: compute-dtw

In all our experiments, each benchmark is run 10 times, and we report the mean runtime and standard deviation (shown in the results tables coloured gray in brackets, separated by $\pm$). Runtimes are reported in milliseconds (to 2 decimal places). When we refer to *speedup* or *slowdown* in reference to a pair of implementations, we mean the ratio of the runtimes for a given benchmark.

5.2.2 RQ1: Overhead of graph construction vs. trace construction. In Table 1, we compare the time taken to evaluate a program in the graph approach (**G-Eval**) of this paper with the time taken to evaluate a program with the trace approach (**T-Eval**) of Perera et al. [2022]. As previously explained, the graph and trace are built as part of the respective program evaluations. Since the graph is a more complex data structure to build compared to the trace, we expect a larger overhead when evaluating a program to a graph compared to evaluating it to a trace.

We report the ratio of graph evaluation time to trace evaluation time in the third column (**Eval-Slowdown**) of Table 1. We see that, as expected, the evaluation of programs to their dynamic dependence graph does indeed incur a higher overhead, being from 3x to 14x slower than their trace-based counterparts.

Table 1. Evaluation Time of Traces Versus Graphs (ms, 2 d.p.)

	T-Eval	G-Eval	Eval-Slowdown
compute-dtw	153.31 (± 12.51)	461.32 (± 16.28)	3.01
edgeDetect	732.38 (± 44.73)	2150.13 (± 109.85)	2.94
emboss	633.36 (± 42.31)	1718.45 (± 30.93)	2.71
gaussian	619.93 (± 48.71)	1714.26 (± 42.52)	2.77
bar-chart-line-chart	2336.85 (± 51.86)	5775.65 (± 131.59)	2.47
stacked-bar-scatter-plot	1212.89 (± 46.21)	7548.93 (± 476.35)	6.22
grouped-bar-chart	89.25 (± 8.36)	1341.60 (± 132.35)	15.03
line-chart	130.02 (± 10.53)	1131.00 (± 14.42)	8.70
stacked-bar-chart	55.92 (± 5.61)	783.03 (± 15.42)	14.00

5.2.3 RQ2: Graph-based demands vs. trace-based demands. In Table 2, columns **T-Demands** and **G-Demands** provide the time taken to compute *demands* with the trace and graph approaches, respectively. The ratio of their performance (graph-based to trace-based) is given in the last column **Bwd-Speedup**.

Overall, we observe a significant speedup in the performance of *demands* using the graph approach, ranging from around 14.6x for the dynamic time warp example **compute-dtw** to more than 80x for **gaussian**. We attribute this performance improvement to the fact that the trace approach requires the system to completely rewind the execution of the original program, in effect evaluating it in reverse. By contrast, in the current graph approach, one only needs to traverse the relevant part of the already-computed graph.

Table 2. Demands Times of Traces Versus Graphs (ms, 2 d.p.)

	T-Demands	G-Demands	Bwd-Speedup
compute-dtw	152.73 (± 9.88)	10.45 (± 0.53)	14.62
edgeDetect	604.96 (± 7.9)	11.90 (± 0.76)	50.84
emboss	504.33 (± 9.12)	7.97 (± 0.66)	63.28
gaussian	511.06 (± 8.33)	6.28 (± 0.77)	81.38
bar-chart-line-chart	1273.98 (± 24.78)	18.50 (± 0.57)	68.86
stacked-bar-scatter-plot	616.63 (± 35.39)	13.97 (± 1.7)	44.14
grouped-bar-chart	74.43 (± 6.22)	1.33 (± 0.27)	55.96
line-chart	114.13 (± 4.84)	1.75 (± 0.21)	65.22
stacked-bar-chart	45.05 (± 5.38)	1.34 (± 0.35)	33.62

5.2.4 RQ3: Implementations of Demanded By. In Table 3, we investigate the performance of *demanded by* in different settings: column **T-DemBy** provides the results for the trace implementation, column **G-DemBy** contains the results for the graph approach using the demBy algorithm in Figure 5a, and column **G-DemBy-Suff** contains the results obtained when computing *demanded*

by by taking the De Morgan dual of suff from Figure 5b. Additionally, in column **Speedup**, we provide the speedup of **G-DemBy** compared to **T-DemBy**, and in column **Suff-Speedup**, we give the speedup of **G-DemBy-Suff** compared to **T-DemBy**.

For all benchmarks apart from **compute-dtw**, we observe that the graph approach using the demBy algorithm in Figure 5a performs better than the other two approaches.

Table 3. Equivalent implementations of Demanded By

	T-DemBy	G-DemBy	Speedup	G-DemBy-Suff	Suff-Speedup
compute-dtw	126.20 (± 4.77)	12272.13 (± 405.55)	0.01	143.66 (± 4.27)	0.88
edgeDetect	696.90 (± 14.85)	265.81 (± 19.08)	2.62	664.32 (± 36.31)	1.05
emboss	603.04 (± 18.03)	244.93 (± 16.31)	2.46	481.51 (± 16.38)	1.25
gaussian	596.78 (± 11.59)	224.27 (± 17.43)	2.66	474.01 (± 11.96)	1.26
bar-chart-line-chart	2078.98 (± 97.24)	19.12 (± 1.23)	108.73	2360.16 (± 47.96)	0.88
stacked-bar-scatter-plot	1004.09 (± 72.66)	31.31 (± 1.33)	32.07	11598.93 (± 778.78)	0.09
grouped-bar-chart	82.37 (± 9.43)	5.00 (± 0.58)	16.47	2146.46 (± 157.97)	0.04
line-chart	113.43 (± 4.87)	2.25 (± 0.52)	50.41	900.31 (± 15.26)	0.13
stacked-bar-chart	42.56 (± 1.85)	3.23 (± 0.97)	13.18	1271.05 (± 131.68)	0.03

5.2.5 Discussion of the results. When comparing the performance of the graph and trace approaches, we observe that the only consistent performance loss for the graph approach is due to the increased cost of evaluating a program to a graph versus evaluating it to a trace (according to RQ1). We argue that this overhead is reasonable, since the graph only needs to be constructed once, and it does not impact subsequent interactions between the user and the system. Additionally, the fast response times for *demands* and *demanded by* compared to the trace approach (as shown by RQ2 and RQ3), translate into quick responses to user queries. This is critical for the interactive exploration of digital media.

Amongst our benchmarks, **compute-dtw** has the worst performance for both *demands* and *demanded by* (although *demands* still shows a considerable speedup when compared to the trace approach—14.62x in Table 2). We hypothesize that this is because program inputs are often required during the computation of other intermediate values in the program (e.g., inputs seq1 and seq2 in computeDTW, 1.25 in Figure 13). Given that each reference to an input results in a dependency edge being added, the dependency graph will have many vertices with a high number of in-edges, where the inputs are the shared source vertices for many of these edges. Thus, the difficulty with examples such as **compute-dtw** may arise from the sharing of values. While our algorithm for computing *demanded by* is graph independent (as described in § 3), these experiments show that the shape of the graph may matter for performance. We defer more detailed discussion to § 7.

Another notable aspect, which is observed for RQ3, is the performance of **G-DemBy-Suff**, which takes the dual of the suff algorithm. In the majority of cases, **G-DemBy-Suff** performs significantly worse than **G-DemBy**. Interestingly, the case where it performs the best is **computeDTW**, where **G-DemBy** suffers from the worst performance. We suspect that the generally bad performance of **G-DemBy-Suff** is at least partially caused by the expensive set complement operations performed before and after applying suff. Further investigation in how the shape of the graph impacts the performance of suff is left to future work, § 7.1.

6 RELATED WORK

6.1 Applying Dynamic Dependence Graphs

We are not the first to consider using dynamic dependence graphs for functional programming languages. For example, the compilation of a functional program to a graph has been used to implement incremental evaluation of programs in [Acar et al. \[2002, 2006\]](#). In contrast to their work, which focusses on efficiently propagating changes for inputs to computations, we are concerned with the implementation of interactive data exploration systems.

For a more generic perspective on program slicing, [Field and Tip \[1998\]](#) consider dynamic dependence tracking in term-rewriting systems. Their approach allows them to perform graphical program slicing on a subset of Pascal, but differs from our work in two ways. Firstly, they are concerned primarily with program slicing of imperative programs as opposed to building interactive visualizations, and secondly, they explicitly represent reduction steps in their dependence graphs. In contrast, we do not represent this sort of information in the graph at all, and instead of representing the semantics as a term-rewriting system, we present ours as an extended big-step relation.

More recently there has been a body of work aimed at using dynamic dependence graphs to assist in program fault localization, but again these are not focussed on interactive visualization systems, but rather on bug finding in programs. For a survey see [Soremekun et al. \[2021\]](#).

6.2 Galois Slicing

Galois connections between lattices offer an appealing mathematical foundation for bidirectional dynamic program analysis, cleanly characterising input-output relationships in terms of demand and sufficiency [[Perera et al. 2012, 2016](#); [Ricciotti et al. 2017](#)]. In these works, Galois connections are applied to the problem of program slicing. The upper adjoint – which characterises forward slicing from inputs to outputs – can be thought of as a *sufficiency* operator, picking out all the outputs which can be computed using only a particular set of inputs; the lower adjoint, which characterises backward slicing, is a *demand* operator, picking out the specific inputs needed to compute a particular set of outputs. [Perera et al. \[2022\]](#) extended this approach to a setting where slices have complements and the slices of a particular program or value form a Boolean lattice; they then showed how the De Morgan dual of the upper adjoint can be used to pick out *related outputs*, namely the other outputs that “compete” with a given set of outputs in the sense of demanding some of the same inputs, and proposed this as a framework for understanding the “brushing and linking” feature commonly used in data visualisation.

However, implementations of Galois slicing efficient enough to support the kind of interactive applications considered here have proved elusive. To compute the two analyses, implementations have relied on a sequential execution record, usually taking the form of a big-step or small-step trace. The forward analysis uses the trace to “replay” the computation, propagating information about resource availability from inputs to outputs; the backwards analysis uses the trace to “rewind” the computation, propagating information about demand from outputs to inputs. This overly sequential approach makes the approach unable to exploit the very independence that makes slicing useful in the first place. Moreover, the backwards analysis makes frequent use of lattice join ($\vee$) to combine slicing information computed along different branches of the computation. Joining closure slices in particular becomes expensive because of the contained environments (which in turn contain closures), and these issues have made the approach prohibitively slow for interactive use. Compared to Galois slicing, our approach of decomposing the problem into a graphical semantics and a pair of analyses over the graph significantly reduces the implementation burden on the developer, since the algorithms for the program analyses are language-agnostic and thus only need to be implemented once.

6.3 Interactive Visualization

We are not the first to consider shifting the burden of interactive visualizations into the language runtime. Reactive Vega, introduced in [Satyanarayan et al. \[2017\]](#), presents a system for building interactivity in visualizations. Their work differs from ours in that their focus is on building new interactions through declarative specifications, as opposed to designing a programming language. Our linking of visualizations is similar in spirit to the work of [North and Shneiderman \[2000\]](#). In that work, the authors design a modular system for replacing visualizations of databases. Their system is modelled by relational database schema, and is designed to facilitate interactions with these modular visualizations, but is limited to data from relational databases, whereas ours can be applied to data of any shape, as long as it is supported by our language. Devising [Livny et al. \[1997\]](#) is another visualization framework for building interactive visualizations of relational database tables. Similar to [North and Shneiderman \[2000\]](#), their work is defined by its usage of relational databases, and the way they create a semantics for queries on visualizations using the model of relational database queries.

7 CONCLUSION AND FUTURE WORK

Our work is motivated by the difficulty and time-consuming nature of producing interactive data visualizations. Visualizations provide a valuable mechanism for communicating research and performing exploratory data analysis, but interactive visualizations are often bespoke solutions for a single application. By shifting the burden into a language runtime, time and effort can be saved by the developer, who needs only to express their data analysis as a program creating the relevant charts, and gets interaction functionality for free.

Prior work that aimed at integrating these features proved to be sensitive to changes in the programming language, and made use of bidirectional analyses which were difficult to implement. By decomposing the system into an evaluator that builds a dependence graph, and a pair of language agnostic graph algorithms, we simplified the implementation of the analyses themselves, and separated the framework from the language.

7.1 Future work

In order to enable a like-for-like comparison, the dependency relation we chose to represent in the graph is the same as that of [Perera et al. \[2022\]](#); here, selections are pushed through evaluation if the selected parts of values are consumed during evaluation - either via pattern matching or the specific implementation provided for a primitive. However, the dependency relation used in that paper was not designed for application in a graph-based setting.

Our results in § 5 (in particular for benchmark **compute-dtw**) suggest that the structure of the graph plays an important role in the performance of our analysis. This raises the question of whether a more appropriate dependency analysis should be implemented specifically for use with our framework, which produces graphs with a somewhat more desirable structure. For example, rather than representing all dependencies as value-to-value dependencies, it may be more useful for edges in the graph to represent causal relationships between steps in the program evaluation. We suggest that a more semantically justified dependency relation could lead to improvements in the scalability of the system, and allow for extended functionality to be implemented.

The analyses in their current form are only sensitive to the *extension* – the I/O relation – of the graph. Two graphs which are *intensionally* different will produce the same conjugate operators if their I/O relation is the same. Using the intensional information present within a graph may allow us to derive more detailed information about the relationships between inputs and outputs. As an example, [Cheney et al. \[2011\]](#) considers the use of dependency analysis to compute *expression*

provenance information about elements of a program. One use of this information is to synthesize a sub-expression describing the derivation of a value at a given point. From the perspective of the graph approach, we suggest that access to intensional information could allow us to take a graph slice and use it to synthesize a subexpression of the original program which can be executed to produce a subset of the outputs.

A final point of inquiry is presented by the relative difference in performance between our two algorithms. It would be interesting to investigate if there are structural properties of the dependence graph that determine which of the implementations of *demand* by will be more efficient. If these structural properties exist, it would also be useful to know how they can be computed, and if they can be computed efficiently.

It follows – if the user is entirely uninterested in the intensional information of their graph – that transformations which elide internal nodes but preserve I/O connectivity, such as the classical $Y\text{-}\Delta$ transform [Truemper 1989], will produce an extensionally equivalent (but faster) analysis. We conjecture that applying this transform to throw away intensional information may be useful in future applications of our work.

REFERENCES

- Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2002. Adaptive functional programming. In *POPL '02: Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon). ACM Press, New York, NY, USA, 247–259. <https://doi.org/10.1145/503272.503296>
- Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2006. Adaptive Functional Programming. *ACM Trans. Program. Lang. Syst.* 28, 6 (nov 2006), 990–1034. <https://doi.org/10.1145/1186632.1186634>
- Hiralal Agrawal and Joseph R. Horgan. 1990. Dynamic program slicing. In *PLDI '90: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation* (White Plains, New York, United States). ACM, New York, NY, USA, 246–256. <https://doi.org/10.1145/93542.93576>
- Donald J Berndt and James Clifford. 1994. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd international conference on knowledge discovery and data mining*. 359–370.
- Nadieh Bremer and Marlieke Ranzijn. 2015. Urbanization in East Asia between 2000 and 2010. <http://nbremer.github.io/urbanization/>.
- James Cheney, Amal Ahmed, and Umut A. Acar. 2011. Provenance as dependency analysis. *Mathematical Structures in Computer Science* 21, 6 (2011), 1301–1337.
- R. DerSimonian and N. Laird. 1986. Meta-analysis in clinical trials. *Control Clin Trials* 7, 3 (Sep 1986), 177–88. [https://doi.org/10.1016/0197-2456\(86\)90046-2](https://doi.org/10.1016/0197-2456(86)90046-2)
- Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The Program Dependence Graph and Its Use in Optimization. *ACM Trans. Program. Lang. Syst.* 9, 3 (jul 1987), 319–349. <https://doi.org/10.1145/24039.24041>
- John Field and Frank Tip. 1998. Dynamic Dependence in Term Rewriting Systems and its Application to Program Slicing. *Information and Software Technology* 40, 11–12 (November/December 1998), 609–636.
- Jeffrey Heer, Maneesh Agrawala, and Wesley Willett. 2008. Generalized Selection via Interactive Query Relaxation. In *ACM Human Factors in Computing Systems (CHI)*. 959–968.
- Ralf Hinze. 2000. Generalizing generalized tries. *Journal of Functional Programming* 10, 4 (2000), 327–351. <https://doi.org/10.1017/S0956796800003713>
- Bjarni Jonsson and Alfred Tarski. 1951. Boolean Algebras with Operators. Part I. *American Journal of Mathematics* 73, 4 (1951), 891–939. <http://www.jstor.org/stable/2372123>
- M. Livny, R. Ramakrishnan, K. Beyer, G. Chen, D. Donjerkovic, S. Lawande, J. Myllymaki, and K. Wenger. 1997. DEVise: integrated querying and visual exploration of large datasets. *SIGMOD Rec.* 26, 2 (jun 1997), 301–312. <https://doi.org/10.1145/253262.253335>
- James Murphy, David Sexton, David Barnett, Gareth Jones, M. Webb, M. Collins, and David Stainforth. 2004. Quantification of modelling uncertainties in a large ensemble of climate change simulations. *Nature* 430 (09 2004), 768–72. <https://doi.org/10.1038/nature02771>
- Chris North and Ben Shneiderman. 2000. Snap-together visualization: a user interface for coordinating visualizations via relational schemata. In *Proceedings of the Working Conference on Advanced Visual Interfaces* (Palermo, Italy) (AVI '00). Association for Computing Machinery, New York, NY, USA, 128–135. <https://doi.org/10.1145/345513.345282>
- Roly Perera, Umut A. Acar, James Cheney, and Paul Blain Levy. 2012. Functional Programs That Explain Their Work. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming* (Copenhagen, Denmark)

- (ICFP '12). ACM, New York, NY, USA, 365–376. <https://doi.org/10.1145/2364527.2364579>
- Roly Perera, Deepak Garg, and James Cheney. 2016. Causally Consistent Dynamic Slicing. In *Concurrency Theory, 27th International Conference, CONCUR '16 (Leibniz International Proceedings in Informatics (LIPIcs))*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany. <https://doi.org/10.4230/LIPIcs.CONCUR.2016.18>
- Roly Perera, Minh Nguyen, Tomas Petricek, and Meng Wang. 2022. Linked Visualisations via Galois Dependencies. *Proc. ACM Program. Lang.* 6, POPL, Article 7 (2022), 29 pages. <https://doi.org/10.1145/3498668>
- Simon Peyton Jones, Richard Eisenberg, and Sebastian Graf. 2022. *Triemaps that match*. Technical Report. <https://simon.peytonjones.org/triemaps-that-match/>
- Fotis Psallidas and Eugene Wu. 2018. Provenance for Interactive Visualizations. In *Workshop on Human-In-the-Loop Data Analytics (HILDA 2018)*. ACM.
- Wilmer Ricciotti, Jan Stolarek, Roly Perera, and James Cheney. 2017. Imperative Functional Programs That Explain Their Work. *Proceedings of the ACM on Programming Languages* 1, ICFP, Article 14 (2017), 28 pages. <https://doi.org/10.1145/3110258>
- Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Trans. Visualization & Comp. Graphics (Proc. InfoVis)* (2017).
- P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data* (Boston, Massachusetts) (*SIGMOD '79*). Association for Computing Machinery, New York, NY, USA, 23–34. <https://doi.org/10.1145/582095.582099>
- Ezekiel Soremekun, Lukas Kirschner, Marcel Böhme, and Andreas Zeller. 2021. Locating faults with program slicing: an empirical analysis. *Empirical Software Engineering* 26, 3 (01 Apr 2021), 51. <https://doi.org/10.1007/s10664-020-09931-7>
- K. Truemper. 1989. On the delta-wye reduction for planar graphs. *Journal of Graph Theory* 13, 2 (1989), 141–148. <https://doi.org/10.1002/jgt.3190130202>

Temporary page!

$\LaTeX$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\LaTeX$ now knows how many pages to expect for this document.